Lecture 13. Model Basics

Model families, fitting, predictions and residuals, formulas, interactions

Contents

13.1 Models and their purpose **2**

13.2 A simple model **2**

13.3 Visualising models **4**

13.4 Formulas and model families **4**

13.4.1 Categorical variables 5

13.4.2 Interactions 5

13.4.3 Transformations 6

13.5 Missing values and other model families **7**

13.6 Selected exercises **7**

13.1 Models and their purpose

Motivation. EDA finds patterns by eye; a model describes a pattern precisely enough to measure it, predict from it and subtract it. This lecture builds the idea of a model from scratch on simulated data, so that what lm() does is no longer a black box.

The goal of a model is a simple low-dimensional summary of a data set: to partition the data into *patterns* and *residuals*. In this course models are tools for exploration, not for inference (p-values, confidence intervals).

Hypothesis generation versus confirmation. Each observation can be used for exploration or for confirmation, not both. Exploration may use an observation as often as you like; confirmation may use it only once. As soon as an observation is used twice you have switched from confirmation to exploration. For confirmatory analysis, split the data in advance into a *training* set (about 60%) for exploration and model fitting, a *query* set (about 20%) to compare models by hand, and a *test* set (about 20%) used only once to test the final model.

A model has two parts:

1. a **family of models** expressing a precise but generic pattern, e.g. the lines $y = a_1 + a_2 x$;
2. a **fitted model**, the member of the family closest to the data, e.g. $y = 3x + 7$.

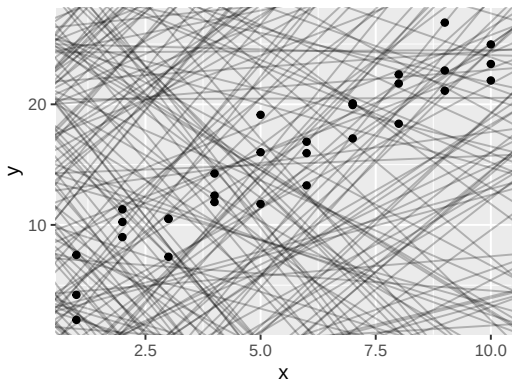
A fitted model is only the closest model from a family; it does not mean the model is "true". "All models are wrong, but some are useful" (G. E. P. Box).

13.2 A simple model

Motivation. Fitting a line seems trivial, but doing it by hand exposes the two ingredients of every fitting method: a measure of distance between model and data, and a search for the parameters that minimise it.

We use `modelr`, whose `options(na.action = na.warn)` makes models warn about dropped missing values. The simulated data `sim1` has a clear linear pattern. Generate many random lines and overlay them:

```
models <- tibble(a1 = runif(250, -20, 40), a2 = runif(250, -5, 5))
ggplot(sim1, aes(x, y)) +
  geom_abline(aes(intercept = a1, slope = a2), data = models, alpha = 1/4) +
  geom_point()
```



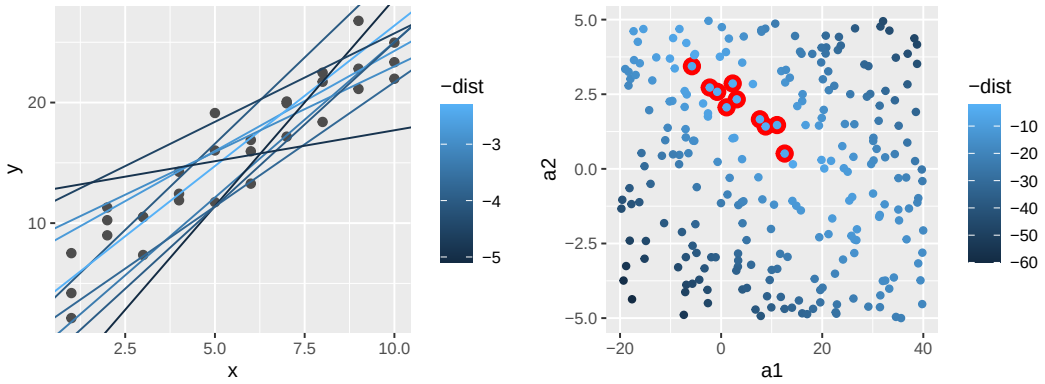
To pick the best line we need the distance between data and model. Write the model as a function of the parameters and data, and measure the distance by the root-mean-squared vertical deviation:

```
model1 <- function(a, data) a[1] + data$x * a[2]
measure_distance <- function(mod, data) {
  diff <- data$y - model1(mod, data)
  sqrt(mean(diff ^ 2))
}
measure_distance(c(7, 1.5), sim1)
```

```
## [1] 2.665212
```

Computing the distance for every random model needs a helper with two scalar arguments, since `map2_dbl()` passes one `a1` and one `a2` at a time:

```
sim1_dist <- function(a1, a2) measure_distance(c(a1, a2), sim1)
models <- models %>% mutate(dist = map2_dbl(a1, a2, sim1_dist))
ggplot(sim1, aes(x, y)) +
  geom_point(size = 2, colour = "grey30") +
  geom_abline(aes(intercept = a1, slope = a2, colour = -dist),
              data = filter(models, rank(dist) <= 10))
ggplot(models, aes(a1, a2)) +
  geom_point(data = filter(models, rank(dist) <= 10), size = 4, colour = "red") +
  geom_point(aes(colour = -dist))
```



The right panel shows each model as a point in parameter space; the ten best are highlighted. A **grid search** replaces the random points by an evenly spaced grid (`expand.grid()`) around the best region and repeats. Better still is a numerical minimiser: `optim()` uses Newton–Raphson-type steps, starting from a point and moving downhill until it cannot improve:

```
best <- optim(c(0, 0), measure_distance, data = sim1)
best$par
```

```
## [1] 4.222248 2.051204
```

For linear models there is a closed form, which `lm()` computes exactly (by QR decomposition rather than iteration):

```
sim1_mod <- lm(y ~ x, data = sim1)
coef(sim1_mod)
```

```
## (Intercept)           x
##    4.220822    2.051533
```

Why the two agree [Supplement]

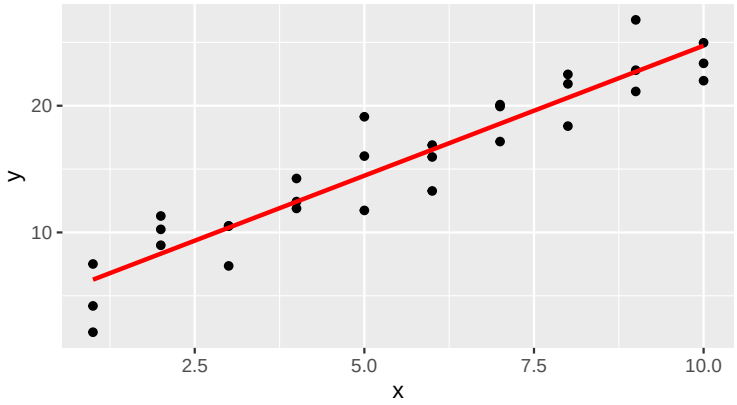
Minimising the root-mean-squared distance is equivalent to minimising the sum of squares $\sum_i (y_i - a_1 - a_2 x_i)^2$, since $\sqrt{\cdot}$ and $1/n$ are increasing. That is the least-squares problem, whose solution is $\hat{a}_2 = \sum (x_i - \bar{x})(y_i - \bar{y}) / \sum (x_i - \bar{x})^2$ and $\hat{a}_1 = \bar{y} - \hat{a}_2 \bar{x}$; `optim()` only approximates it numerically.

13.3 Visualising models

Motivation. Coefficients are hard to interpret once models grow. A more general approach looks at what a model predicts and what it leaves unexplained, and this works for every kind of model.

Predictions. Build an evenly spaced grid covering the region of the data with `data_grid()`, then add the model's predictions with `add_predictions()`:

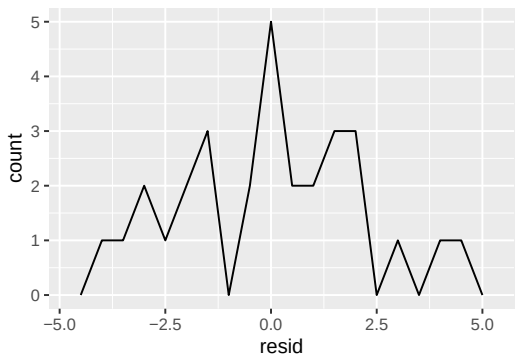
```
grid <- sim1 %>% data_grid(x) %>% add_predictions(sim1_mod)
ggplot(sim1, aes(x)) +
  geom_point(aes(y = y)) +
  geom_line(aes(y = pred), data = grid, colour = "red", linewidth = 1)
```

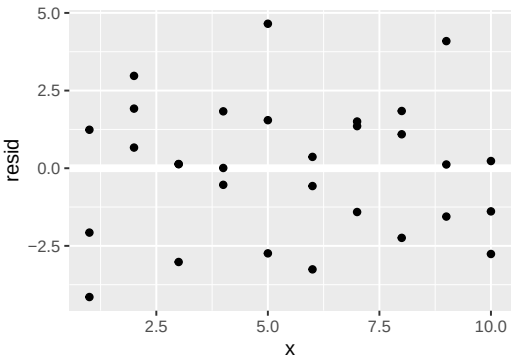


This works for any model, not just lines, and shows what the model has captured.

Residuals are the distances between observed and predicted values; they show what the model has missed. `add_residuals()` adds them to the original data (residuals need the observed `y`, so a grid will not do). A residual plot against `x` should look like random noise if the model captured the pattern:

```
sim1 <- sim1 %>% add_residuals(sim1_mod)
ggplot(sim1, aes(resid)) + geom_freqpoly(binwidth = 0.5)
ggplot(sim1, aes(x, resid)) + geom_ref_line(h = 0) + geom_point()
```





13.4 Formulas and model families

Motivation. R's formula notation is a compact language for specifying model families. Seeing how a formula becomes a design matrix explains what each model can and cannot fit.

A formula `y ~ x` is translated into $y = a_1 + a_2 x$. `model_matrix()` shows the translation: the columns of the design matrix, one per coefficient, with an intercept column of ones unless `- 1` removes it.

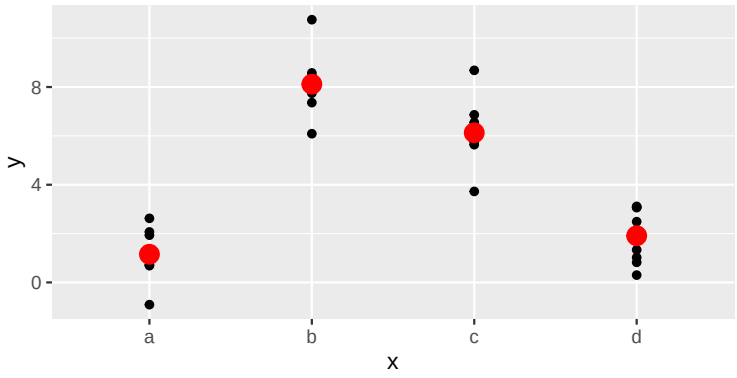
```
df <- tribble(~y, ~x1, ~x2,  4, 2, 5,  5, 1, 6)
model_matrix(df, y ~ x1 + x2)
```

```
## # A tibble: 2 x 3
##   `(Intercept)`    x1    x2
##           <dbl> <dbl> <dbl>
## 1             1     2     5
## 2             1     1     6
```

13.4.1 Categorical variables

For a categorical predictor `y ~ sex`, R creates an indicator column `sexmale` (1 for male, 0 otherwise); a column for `sexfemale` would be redundant, being one minus the other. With a categorical `x`, the fitted model predicts the mean of each category:

```
mod2 <- lm(y ~ x, data = sim2)
grid <- sim2 %>% data_grid(x) %>% add_predictions(mod2)
ggplot(sim2, aes(x)) +
  geom_point(aes(y = y)) +
  geom_point(data = grid, aes(y = pred), colour = "red", size = 4)
```

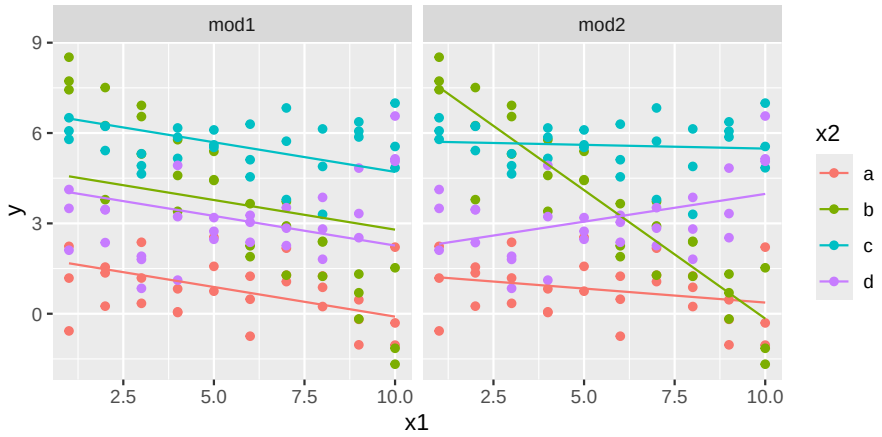


Predicting for a level the model never saw (`tibble(x = "e")`) is an error.

13.4.2 Interactions

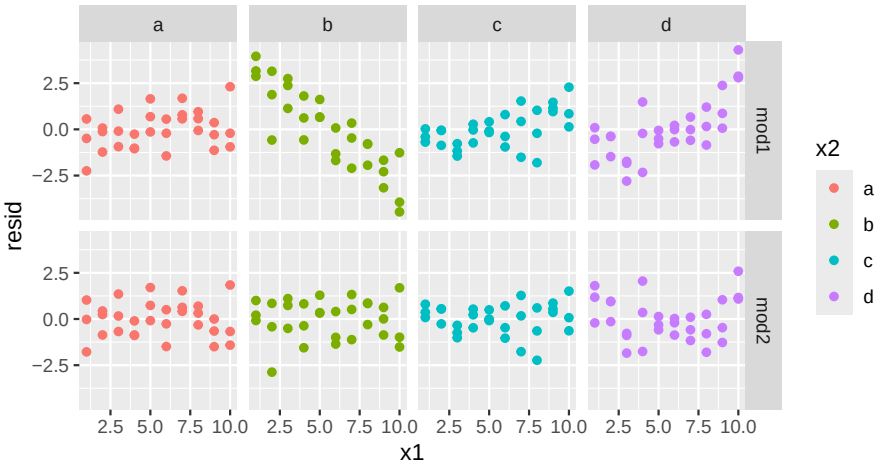
`sim3` has a continuous `x1` and a categorical `x2`. Two models are possible: `y ~ x1 + x2` fits parallel lines (same slope, different intercepts), while `y ~ x1 * x2` also includes the interaction `x1:x2`, so each line has its own slope. `gather_predictions()` adds predictions from several models with a `model` column:

```
mod1 <- lm(y ~ x1 + x2, data = sim3)
mod2 <- lm(y ~ x1 * x2, data = sim3)
grid <- sim3 %>% data_grid(x1, x2) %>% gather_predictions(mod1, mod2)
ggplot(sim3, aes(x1, y, colour = x2)) +
  geom_point() +
  geom_line(data = grid, aes(y = pred)) +
  facet_wrap(~ model)
```



Which is better? The residuals of `mod1` still show clear patterns in groups `b`, `c`, `d`; those of `mod2` look random. In EDA, inspecting residuals answers the question; formal comparison is a topic of inference.

```
sim3 <- sim3 %>% gather_residuals(mod1, mod2)
ggplot(sim3, aes(x1, resid, colour = x2)) +
  geom_point() +
  facet_grid(model ~ x2)
```



With two continuous variables (`sim4`), `y ~ x1 * x2` fits $y = a_0 + a_1 x_1 + a_2 x_2 + a_{12} x_1 x_2$. A grid built with `seq_range(x1, 5)` (five evenly spaced values over the range) lets us compare the models; the interaction shows as slopes that change with the other variable.

13.4.3 Transformations

Transformations can be written inside the formula, e.g. `log(y) ~ sqrt(x1) + x2`. Operators that have a formula meaning (`+`, `*`, `^`, `-`) must be wrapped in `I()`: `y ~ x + I(x^2)` is the quadratic $a_1 + a_2 x + a_3 x^2$, whereas `y ~ x^2 + x` means `y ~ x`. Linear models can thus fit nonlinear curves, as long as the model is linear in the coefficients. `poly(x, 2)` gives orthogonal polynomials, and `splines::ns(x, df)` natural splines, which behave better outside the data range: polynomials diverge quickly beyond the data.

```
df <- tribble(~y, ~x,  1, 1,  2, 2,  3, 3)
model_matrix(df, y ~ x + I(x^2))
```

```
## # A tibble: 3 x 3
##   `(Intercept)`     x `I(x^2)`
##           <dbl> <dbl>    <dbl>
## 1             1     1        1
## 2             1     2        4
## 3             1     3        9
```

Extrapolation [Supplement]

Every model is fitted to the range of the data. Outside it the model's behaviour is dictated only by its family: a polynomial shoots to $\pm\infty$, a natural spline continues linearly. Neither is evidence about the data there, so predictions far outside the observed range should not be trusted.

13.5 Missing values and other model families

Rows with missing values cannot be used in the fit; `lm()` drops them silently by default, and with `na.action = na.warn` it warns. `nobs(mod)` gives the number of observations actually used.

Linear models assume a continuous residual with the same variance everywhere. Many extensions exist:

- generalised linear models, `glm()`, for binary or count responses;
- generalised additive models, `mgcv::gam()`, which fit smooth functions `y ~ s(x)`;
- penalised models such as `glmnet`, which shrink coefficients towards zero;
- robust models, `MASS::rlm()`, which reduce the influence of outliers;
- trees and forests (`rpart`, `randomForest`, `xgboost`).

They are fitted and explored with the same predictions-and-residuals approach.

13.6 Selected exercises

Motivation. The book's exercises for this chapter ask what can go wrong in fitting and what a formula really specifies. The answers come from two facts: a model is a set of fitted values, which different parameterisations can describe equally well, and least-squares residuals are orthogonal to every column of the design matrix.

Exercise 13.1 (R4DS Exercise 23.2.1.3)

What is the problem with optimising the three-parameter model $f(x; a) = a_1 + a_2 x + a_3$ by minimising the root-mean-squared distance?

Solution.

Step 1. The prediction depends on a only through $(a_1 + a_3, a_2)$. Hence the distance $d(a)$ satisfies $d(a_1 + t, a_2, a_3 - t) = d(a)$ for every t.

Step 2. By the notes ("Why the two agree") the minimum over (c, a_2), with $c = a_1 + a_3$, is attained at the least-squares line, uniquely when x is not constant. So the minimisers of d are the whole line $\{(c - t, \hat{a}_2, t) : t \in \mathbb{R}\}$, not a single point: the parameters are not *identifiable*.

Step 3. A numerical optimiser such as `optim()` will stop at some point of this line that depends on the starting value, and different runs report different a_1, a_3 although all describe the same fitted line. The model family is fine; the parameterisation is redundant. R's formula interface avoids this by never creating two intercept columns. ■

Exercise 13.2 (R4DS Exercise 23.4.5.1)

For `sim2`, whose predictor `x` has levels a, b, c, d, compare `y ~ x` with `y ~ x - 1`. What happens to the model equation and to the predictions?

Solution.

Step 1. (Equations.) With an intercept the design matrix has columns 1, x_b, x_c, x_d (indicators of levels b, c, d), so $\hat{y} = a_1 + a_2 x_b + a_3 x_c + a_4 x_d$. Without the intercept R keeps all four indicators, $\hat{y} = a_a x_a + a_b x_b + a_c x_c + a_d x_d$.

Step 2. (Same column space.) Since $x_a + x_b + x_c + x_d = 1$, the column 1 is a combination of the four indicators and $x_a = 1 - x_b - x_c - x_d$ is a combination of the first design's columns. The two designs span the same space, and least squares projects y onto that space, so the fitted values, the residuals and all predictions are identical.

Step 3. (Coefficients.) Only the meaning changes: without the intercept $\hat{a}_\ell$ is the mean of group ℓ; with it, $\hat{a}_1$ is the mean of group a and $\hat{a}_2, \hat{a}_3, \hat{a}_4$ are differences from group a. ■

Exercise 13.3 (R4DS Exercise 23.4.5.2)

Which columns does `model_matrix()` create for `y ~ x1 * x2` in `sim3` (continuous `x1`, `x2` with levels a, b, c, d) and in `sim4` (both continuous)? Why is `*` a good shorthand?

Solution.

Step 1. `x1 * x2` expands to `x1 + x2 + x1:x2`. In `sim3` the categorical `x2` becomes the indicators x_{2b}, x_{2c}, x_{2d}, and the interaction multiplies `x1` by each: the columns are intercept, x_1, x_{2b}, x_{2c}, x_{2d}, $x_1 x_{2b}$, $x_1 x_{2c}$, $x_1 x_{2d}$, eight in all, i.e. a separate intercept and a separate slope for each of the four groups.

Step 2. In `sim4` the columns are intercept, x_1, x_2, $x_1 x_2$, giving $y = a_0 + a_1 x_1 + a_2 x_2 + a_{12} x_1 x_2$, in which the slope in x_1 is $a_1 + a_{12} x_2$.

Step 3. An interaction alone is rarely meaningful: without the main effects, the model would force the slope in x_1 to vanish when $x_2 = 0$ (or in the reference group), a constraint that depends on the arbitrary origin or coding of x_2. Writing `*` always includes the lower-order terms, so the fit does not depend on such choices. ■

Exercise 13.4 (what a residual plot can show)

Let $\hat{a}$ be the least-squares fit of a model whose design matrix X contains an intercept column and a column x, and let $r = y - X\hat{a}$. Show that the residuals have mean 0 and zero sample correlation with x (and with every other column of X). What does this imply for a plot of r against x, and against a variable not in the model?

Solution [Supplement].

Step 1. The least-squares problem $\min_a \|y - Xa\|^2$ has gradient $-2X^\mathsf{T}(y - Xa)$, which vanishes at the minimiser, so $X^\mathsf{T} r = 0$: r is orthogonal to every column of X.

Step 2. Orthogonality to the intercept column gives $\sum_i r_i = 0$, so $\bar{r} = 0$. Orthogonality to x then gives $\sum_i (x_i - \bar{x})(r_i - \bar{r}) = \sum_i x_i r_i - \bar{x} \sum_i r_i = 0$, so the sample covariance, and hence the correlation,

of r and x is 0.

Step 3. A residual plot against an included x therefore never shows a linear trend; any pattern it shows (curvature, changing spread, clusters) is something the linear term cannot capture. Against a variable not in the model the residuals can have any correlation, and a trend there is direct evidence that this variable should be added. This is how the residuals of `mod1` for `sim3`, plotted by group, revealed the missing interaction columns $x_1 x_{2b}, x_1 x_{2c}, x_1 x_{2d}$. ■